

CSCE 46103/56103 Artificial Intelligence – Fall 2026

Assignment 3: Pac-Man Adversarial Search

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

September 24, 2026

Assignment Overview

Implement three search functions

In `adversarial_search.py`:

- ▶ `minimax_search`
- ▶ `alpha_beta_search`
- ▶ `monte_carlo_tree_search`

Compare their decisions and search effort in the supplied Pac-Man game.

Your evidence and deliverables

Independent tests, required hand traces, reproducible experiments, and a report of at most eight pages.

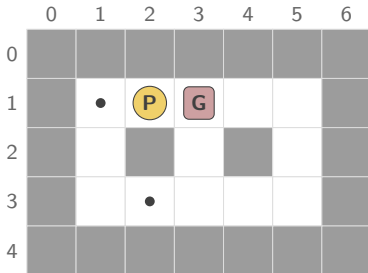
Supplied for you

Game engine, cutoff evaluator, search helpers, random baseline, replay tools, and five levels.

Individual assignment, 100 points.

Python 3.10+ and the standard library.

Pac-Man Basics



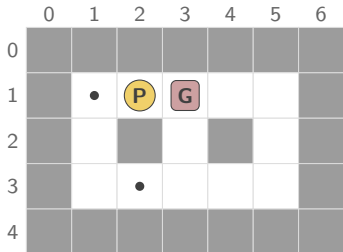
Supplied danger.json

- ▶ Pac-Man (P) collects pellets.
- ▶ The ghost (G) tries to catch Pac-Man.
- ▶ Walls block movement.
- ▶ Collect every pellet before capture or the move limit.

Course adaptation

Pac-Man is MAX and the ghost is MIN.
Both see the complete state, and every move has a deterministic result.

Game State and Legal Actions



Coordinates: (row, column)

State field	Initial value
Pac-Man / ghost	(1, 2) / (1, 3)
Remaining pellets	{(1, 1), (3, 2)}
Player to move	MAX
Remaining plies	24

Walls belong to the fixed maze.

Canonical action order

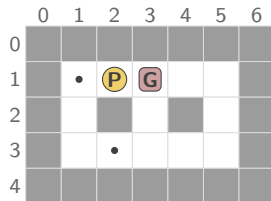
UP, RIGHT, DOWN, LEFT, WAIT

Omit blocked directions. Here Pac-Man can choose RIGHT, LEFT, WAIT.

Moving onto the opponent is legal and causes capture. Terminal states have no actions.

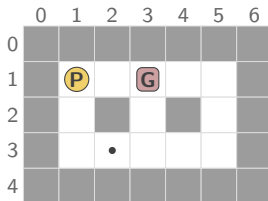
One Move, One Ply

Before the move



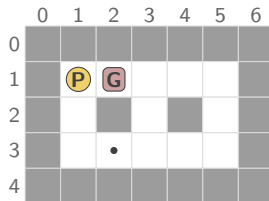
MAX to move
24 plies remain

Pac-Man takes LEFT



MIN to move
23 plies remain

Ghost takes LEFT



MAX to move
22 plies remain

Pac-Man collects the pellet at (1,1). The ghost never collects pellets.

Two plies normally give each player one turn.

Capture, Completion, and Timeout

1. Capture



Same position

$$U = -1$$

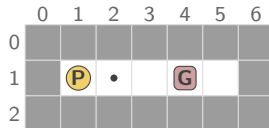
2. Completion



No pellets remain

$$U = +1$$

3. Timeout



No plies remain

$$U = -1$$

These teaching states all have zero remaining plies.

Capture, Completion, and Timeout

1. Capture



Same position

$$U = -1$$

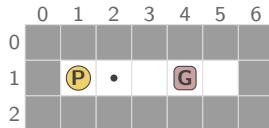
2. Completion



No pellets remain

$$U = +1$$

3. Timeout



No plies remain

$$U = -1$$

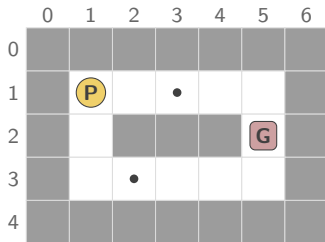
These teaching states all have zero remaining plies.

Check outcomes in this order

Capture overrides collecting the last pellet. Without capture, collecting the last pellet on the final allowed ply succeeds.

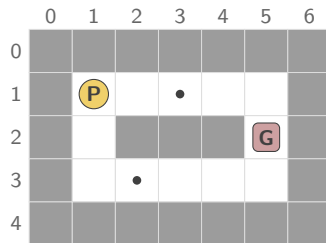
Waiting and Cycles

Initial state



MAX to move, 32 plies remain

After WAIT, WAIT



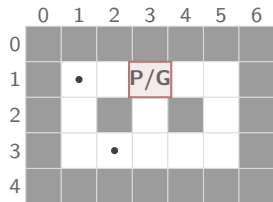
MAX to move, 30 plies remain

The positions match. The states differ in remaining plies.

Waiting and cycles still consume the finite game budget.

An Immediate-Danger Decision

RIGHT



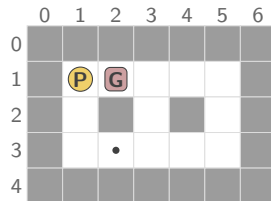
Immediate capture
 $U = -1$

WAIT, then ghost LEFT



Ghost can capture
 $U = -1$

LEFT, then ghost LEFT



Pac-Man survives this reply
One pellet remains

Looking ahead includes the opponent's reply.
Surviving these two plies does not establish a forced win.

Search Opponent and Gameplay Opponent

Inside your search

Consider every legal ghost action as an adversarial response.

The search interface must also work from a MIN root and on tiny abstract test games.

Inside the supplied experiment runner

Your algorithm controls Pac-Man. The supplied chasing policy controls the ghost.

The ghost minimizes maze distance to Pac-Man, breaking ties in canonical order.

Each Pac-Man turn starts a new search from the current state.

A winning replay shows success against the supplied policy.

It does not establish a forced win against optimal defense.

Three Different Limits

Limit	What it limits	Example
Episode max_plies	Moves in the entire played game	24 in danger.json
Search depth	Moves explored from the current root	Depth 4: four plies
MCTS iterations	Completed selection-to-update cycles per search	200 sampled iterations

Supplied Utility and Cutoff Evaluation

Terminal utility for MAX: completion +1, capture or timeout -1.

$$E(s) = 0.6p + \frac{0.2}{1 + d_{\text{food}}} + \frac{0.2d_{\text{ghost}}}{1 + d_{\text{ghost}}} - 0.5$$

p Fraction of initial pellets already collected

d_{food} Shortest maze distance to a remaining pellet

d_{ghost} Shortest maze distance between the players

Keep the supplied evaluator unchanged

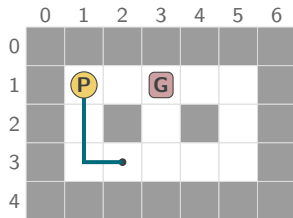
Check terminal status before the depth cutoff. Use $E(s)$ only for a nonterminal cutoff.

MCTS rollouts reach terminal utility.

One Pac-Man Evaluation

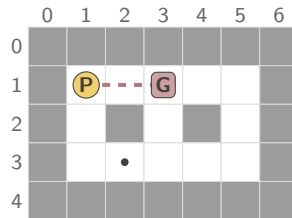
Pac-Man takes LEFT and collects one of two pellets, so $p = 1/2$.

Nearest remaining pellet



$$d_{\text{food}} = 3$$

Distance between players

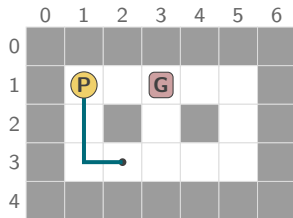


$$d_{\text{ghost}} = 2$$

One Pac-Man Evaluation

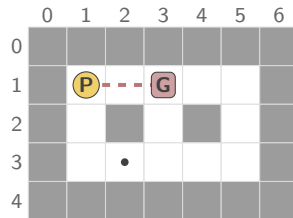
Pac-Man takes LEFT and collects one of two pellets, so $p = 1/2$.

Nearest remaining pellet



$$d_{\text{food}} = 3$$

Distance between players



$$d_{\text{ghost}} = 2$$

$$E(s) = 0.3 + \frac{0.2}{4} + \frac{0.4}{3} - 0.5 = -\frac{1}{60} \approx -0.016667.$$

Distances respect walls. This cutoff estimate does not prove a loss.

Minimax and Alpha–Beta Requirements

Both implementations

- ▶ Handle either MAX or MIN at the root.
- ▶ Count depth in individual moves and check terminal status first.
- ▶ Reverse every action list when requested.
- ▶ Keep the first action on exact value ties.

Additional alpha–beta rules

- ▶ Prune when $\alpha \geq \beta$, including equality.
- ▶ Return the best child value encountered, without clipping to the bounds.
- ▶ Generate no skipped successors. Count a cutoff only when it skips actions.

At the same depth and order, both methods must return the same root action and value.

MCTS Requirements

Tree decisions and returns

- ▶ Expand the first untried action. Resolve UCT ties in canonical order.
- ▶ Use the parent's player for the UCT exploitation sign. Keep the exploration term positive.
- ▶ Add the same MAX-perspective return at the stored node and every ancestor, including the root.

Follow the supplied contract for random calls and search accounting.

Budget and final recommendation

- ▶ Complete exactly the requested iterations for a nonterminal root.
- ▶ Simulate to terminal utility. Store expanded nodes, leaving rollout states temporary.
- ▶ Return the most-visited root action and its mean return. Resolve ties canonically.

Supplied Interfaces and Editable Files

A generic game interface

Initial state, player to move, legal actions, successor state, terminal status, utility, and cutoff evaluation.

Use the interface without inspecting maze internals. Instructor tests can use tiny deterministic trees.

Permitted code changes

Implement the TODOs and add private helpers in `adversarial_search.py`. Add tests in `tests/test_student.py`.

One SearchContext per call

Use its transition, evaluation, accounting, random, and trace helpers. Return through `context.finish`.

Preserve supplied interfaces and files. Use the validation helpers and reject invalid inputs with `ValueError`.

No visited set, transposition table, or memoization. Stop by the specified limits and terminal rules.

Reproducible Random Calls

A fixed seed reproduces a run only when the random-call protocol also matches.

Deterministic tree decisions

Minimax and alpha–beta use no draws.

MCTS expands the first untried action and resolves UCT ties canonically.

Every rollout move

Draw exactly one index with the context's private random source, even if only one action is legal.

Example scripted rollout

Actions	Index	Chosen
[a, b]	1	b
[c]	0	c
terminal	no draw	stop

Scripted integers are action indices. They are not seeds.

Backpropagation and final action selection consume no draws.

Search Effort and Decision Frames

Event	What counts	Supplied helper
Generated transition	One successor call, including rollouts	successor; rollout=True for rollout moves
Evaluated leaf	One visited recursive leaf or one MCTS terminal return	evaluate
Recursive backup	Record action and returned value	record_backup
Actual cutoff	Count only if remaining actions are skipped	record_prune, then backup
Stored MCTS node	Root or expanded child	new_node
Completed iteration	Update path, then record statistics	record_iteration

Terminal root: action `None`, exact utility, one evaluation, zero successors. MCTS also has zero iterations and stored nodes.

MCTS iteration returns use `record=False`.
Following a stored edge adds no successor call.
Internal backups add no leaf evaluation.

Pre-Run Traces and Independent Tests

Before implementing a TODO

Complete the required minimax, alpha-beta, three-iteration MCTS, and opponent-aware UCT traces in the report.

First student test

Use a known numerical answer. Reach a TODO and record its initial `NotImplementedError`.

Tests should distinguish defects

- ▶ Maximizing at a MIN node
- ▶ Evaluating a terminal as a cutoff
- ▶ Missing an equality cutoff
- ▶ Using incorrect UCT or return signs
- ▶ Storing rollout states or choosing by mean

The public suite passes with all three TODOs unimplemented.

Independently check actions, values, counters, traces, ties, and invalid inputs.

Replays and Verification

Start with the supplied random baseline

From the assignment directory:

```
python demo.py --algorithm random --level levels/danger.json \  
    --seed 7 --output results.html
```

Inspect the offline HTML

Step through game moves and recorded search decisions. Connect chosen actions with values, pruning events, or root-child statistics.

Python makes the decisions. The browser displays the recorded replay.

Verify your implementations

Inspect danger, trap, and cycle replays. Repeat identical inputs to check identical results and traces.

Run public and student tests separately. Review code paths after tests pass, and record what you checked.

The Required Experiment Matrix

After implementing all TODOs: `python run_experiments.py`

Method	Settings on each of five levels	Games
Minimax and alpha-beta	Depths 2, 4, 6, both orders	60
MCTS	50, 200, 800 iterations, seeds 0, 1, 2	45
Random	Seeds 0, 1, 2	15
Total		120

Levels: danger, trap, routes, cycle, larger.

Check the outputs

Inspect `summary.csv`, `index.html`, and individual replays. Every completed game must reach a terminal outcome. Rows marked `todo` are incomplete.

Required Comparisons

Minimax and alpha-beta

At identical depths and orders, verify matching action sequences and per-decision values. Compare actual search counts.

Use the first decision to compare move orders from the same initial state. Tied choices may change later play.

Gameplay outcomes and computational work

Report wins, pellet progress, game length, and effort. For randomized runs, include means and sample standard deviations of numerical metrics. Compare total work and mean work per decision.

MCTS across seeds

At each level's first decision, report chosen-action frequencies and the mean and sample standard deviation of selected-action means.

Compare iteration budgets across seeds. More iterations need not improve every run.

Report and Verification Evidence

At most eight pages

Use the supplied NeurIPS 2026 style. The limit includes traces, figures, tables, and references.

- ▶ Required pre-run traces and game model
- ▶ Algorithm choices and comparisons
- ▶ Gameplay and replay observations
- ▶ Independent test and the defect it detects

Reproducible evidence

Record commands and observed outcomes, source review, and review of the exact planned submission.

Include Python version, dependencies, one input assumption, and one known limitation.

Aggregate results by level, method, and settings. Explain selected games without reproducing every run.

Compile `report/report.pdf` from your completed report source.

Submission and Grading

Blackboard submission paths

- ▶ `adversarial_search.py`
- ▶ `tests/test_student.py`
- ▶ `report/report.tex`
- ▶ `report/report.pdf`
- ▶ `AI_USE.md`

Component	Points
Minimax	20
Alpha-beta	20
MCTS	25
Student tests	20
Report and verification	15
Total	100

Complete AI-use declaration and include reasonably available AI-use and verification records.

Exclude generated replays, experiment output, caches, and unchanged supplied files outside the submission list.

Thank You

Any questions?